BusBooking

Online Bus Ticket Reservation System

Software Design Document

Field	Details
Document Title	BusBooking – Online Bus Reservation (Software Design Document)
Version	1.0
Prepared By	Development Team
Date	April 11, 2026
Document Status	Initial Draft
Classification	Internal Use Only

Table of Contents

1. Problem Statement
2. Our Approach
3. Technology Stack
4. System Architecture
5. Frontend Implementation
6. Backend Implementation
7. Database Implementation (MySQL)
8. API Design
9. Challenges Faced & Solutions
10. Conclusion

1. Problem Statement

The Indian intercity bus travel industry serves millions of passengers daily, yet the booking experience remains fragmented and inefficient. Travelers face several persistent challenges:

- Lack of a centralized platform to compare buses across multiple operators, leading to suboptimal choices in terms of price, comfort, and timing.
- No real-time seat availability visibility – passengers often arrive at bus stands only to find that their preferred seats or buses are fully booked.
- Manual, paper-based ticketing systems that are prone to overbooking, data loss, and fraud, resulting in poor customer trust.
- Absence of secure digital payment integration, forcing users to rely on cash transactions that offer no refund protection or audit trail.
- No post-booking management capabilities such as e-ticket generation, booking history, or digital PNR tracking.

These inefficiencies result in revenue leakage for operators, frustration for passengers, and an overall decline in service quality. There is a clear need for a modern, full-stack web application that digitizes the entire bus booking lifecycle – from search and seat selection to payment processing and ticket management – while providing a premium, intuitive user experience.

2. Our Approach

We adopted a systematic, user-centered development methodology to build BusBooking. Our approach was guided by industry best practices and a focus on delivering a production-grade application.

2.1 Research & Requirements Gathering

- Conducted competitive analysis of existing platforms (RedBus, AbhiBus, MakeMyTrip Buses) to identify UX patterns and feature gaps.
- Defined user personas covering frequent travelers, occasional users, and first-time digital bookers.
- Mapped the complete user journey from search to post-booking, identifying pain points at each stage.

2.2 Architecture-First Design

- Adopted a decoupled client-server architecture with a React SPA frontend and Flask REST API backend, enabling independent development and deployment.
- Designed the database schema upfront using ER modeling to ensure referential integrity and query efficiency.
- Established API contracts (request/response schemas) before implementation to enable parallel frontend-backend development.

2.3 Iterative Development

- Followed an incremental build approach: Core booking flow first, then authentication, then payment, then polish.
- Implemented responsive design from the start using Tailwind CSS utility classes and mobile-first breakpoints.
- Added progressive enhancements like animated transitions (Framer Motion), glassmorphism UI effects, and real-time seat availability indicators.

2.4 Security & Reliability

- Implemented JWT-based stateless authentication with token expiry and secure password hashing (Werkzeug).
- Added input validation on both client and server sides to prevent injection attacks and malformed data.
- Integrated Razorpay payment gateway with proper error handling, retry logic, and payment verification callbacks.
- Implemented email notifications for booking confirmations, login alerts, and password reset flows using asynchronous background tasks.

3. Technology Stack

The system leverages a modern full-stack architecture optimized for performance, developer productivity, and scalability.

Layer	Technology	Purpose
Frontend	React 18 + Vite	Component-based SPA with fast HMR
Styling	Tailwind CSS	Utility-first responsive design system
Animations	Framer Motion	Fluid page transitions and micro-interactions
Routing	React Router v6	Client-side navigation with nested routes
Icons	Lucide React	Consistent, tree-shakeable icon library
Backend	Flask (Python)	Lightweight RESTful API framework
ORM	SQLAlchemy	Pythonic database abstraction layer
Database	MySQL	Relational data storage with ACID compliance
Authentication	JWT (PyJWT)	Stateless token-based auth with expiry
Payments	Razorpay SDK	PCI-DSS compliant payment processing
Email	SMTP (Async)	Transactional emails via background threads
Security	Werkzeug	Password hashing with salt (PBKDF2)

4. System Architecture

BusBooking follows a three-tier client-server architecture with clear separation of concerns between the presentation layer, business logic layer, and data access layer.

4.1 High-Level Architecture

Component	Details
Client (Browser)	React SPA served via Vite dev server; communicates with backend via Axios HTTP client
API Gateway	Flask Blueprint-based routing with /api prefix; handles CORS, auth middleware, request validation
Business Logic	Route handlers implementing search, booking, auth, payment verification, and email dispatch
Data Layer	SQLAlchemy ORM mapping to MySQL tables; connection pooling and transaction management
External Services	Razorpay (payments), SMTP (emails), JWT (auth tokens)

4.2 Request Flow

- User interacts with React UI → Axios sends HTTP request with JWT in Authorization header
- Flask receives request → login_required decorator validates token → extracts user_id
- Route handler executes business logic → SQLAlchemy queries/mutations on MySQL
- JSON response returned → React updates UI state → User sees result

4.3 Authentication Flow

- Registration: Client sends base64-encoded password → Server hashes with Werkzeug → Stores in DB → Welcome email sent
- Login: Credentials verified → JWT generated with user_id claim → Token returned → Login alert email dispatched
- Password Reset: Timed token generated via itsdangerous → Reset link emailed → Token validated within 15-minute window

5. Frontend Implementation

The frontend is a single-page application built with React 18, featuring six core pages with a premium, glassmorphism-inspired design language.

Page / Component	Description & Key Features
Home	Hero section with animated gradient orbs, city autocomplete search with keyboard navigation, popular routes grid with split-image cards, promotional offers carousel, quick date selectors (Today/Tomorrow)
Buses (Search Results)	Real-time bus listing with filter sidebar (AC/Non-AC, Sleeper/Seater, departure time), multi-criteria sorting (price, rating, departure), animated card transitions, availability badges
Seats	Interactive dual-deck seat map (Lower/Upper) with color-coded availability, 3-step stepper flow (Select Seats → Boarding/Drop Points → Passenger Details), gender-based seating rules, bus image slider
Payment	Secure checkout with Razorpay integration, detailed fare breakdown (base fare, GST, platform fee), success/error state animations, automatic redirect to booking confirmation
My Bookings	Ticket-style booking cards with PNR display, PDF download via html2canvas + jsPDF, passenger details grid, hover-reveal action bar
Auth Page	Unified login/signup/forgot/reset interface with split-panel layout, base64 password encoding, device detection for login alerts, URL-based mode switching

5.1 Design Highlights

- Glassmorphism UI: Semi-transparent backgrounds with backdrop blur effects on search bars and filter panels
- Micro-animations: Framer Motion spring physics for card entrances, staggered list reveals, and hover transformations
- Responsive Layout: Mobile-first approach with Tailwind breakpoints (sm/md/lg) ensuring usability across all device sizes
- Consistent Color System: Red-600 primary accent, Slate-900 for text hierarchy, gradient cards for visual interest

6. Backend Implementation

The backend is built with Flask, organized using Blueprints for modular route registration. Key implementation details include:

- Blueprint-based routing: All API endpoints registered under /api prefix for clean URL namespacing
- Middleware pattern: Custom login_required decorator extracts and validates JWT from Authorization header before route execution
- Input validation: Server-side checks for email format (regex), date validity (future dates only), passenger age bounds (5–100), and required field presence
- Seat generation: Dynamic seat creation via get_or_create_seats utility – generates seat layouts on first request for a bus-date combination
- Gender-based seating rules: Adjacent seat pairs enforce gender compatibility (female-booked seats restrict neighboring seat to female only)
- Asynchronous email dispatch: Booking confirmations, login alerts, and password reset emails sent via background threads to avoid blocking API responses
- Secure token management: itsdangerous library for timed password reset tokens with 15-minute expiry; PyJWT for session tokens

7. Database Implementation (MySQL)

The database schema is designed around normalized relational tables with foreign key constraints ensuring referential integrity.

Table	Key Columns	Purpose
users	id, name, email (unique), password (hashed)	User accounts with secure credential storage
routes	id, source, destination	Available travel routes between cities
buses	id, route_id (FK), travels_name, bus_type, is_ac, price, rating, total_seats	Bus inventory with operator details and pricing
seats	id, bus_id (FK), journey_date, seat_no, is_available, price, seat_type, deck, gender	Per-journey seat availability with type/deck metadata
bookings	id, pnr (unique), user_id (FK), bus_id, source, destination, journey_date, total_amount	Confirmed reservations with PNR tracking
passengers	id, booking_id (FK), name, age, gender, seat_no	Individual traveler details per booking

route_stops	id, city, stop_name, address	Boarding and dropping point locations per city

7.1 Key Relationships

- Users → Bookings: One-to-Many (a user can have multiple bookings)
- Routes → Buses: One-to-Many (a route can have multiple buses operating on it)
- Buses → Seats: One-to-Many (seats generated per bus per journey date)
- Bookings → Passengers: One-to-Many (each booking contains one or more passengers)

8. API Design

The backend exposes a RESTful API with consistent JSON request/response formatting. All endpoints follow the /api prefix convention.

Method	Endpoint	Auth	Description
GET	/api/search?source=X&destination=Y&date=Z	Public	Search available buses
GET	/api/bus/:id	Public	Get bus details
GET	/api/bus/:id/seats?date=Z	Public	Get seat layout
POST	/api/book	JWT Required	Create booking
POST	/api/my-bookings	JWT Required	Get user bookings
POST	/api/auth/register	Public	User registration
POST	/api/auth/login	Public	User login
POST	/api/auth/forgot	Public	Request password reset
POST	/api/auth/reset-password	Public	Reset password
GET	/api/getRouteStops	Public	Boarding/dropping points
GET	/api/getBusImages	Public	Random bus gallery

8.1 Error Handling Convention

- All errors return JSON with an "error" key and appropriate HTTP status codes (400, 401, 404, 500)
- Input validation errors provide field-specific messages (e.g., "Invalid date. Must be today or a future date.")
- Authentication failures return 401 with clear token-related messages ("Token missing", "Invalid or expired token")

9. Challenges Faced & Solutions

Challenge	Impact	Solution
Seat layout generation at scale	Generating 30+ seats per bus per date would be slow if done eagerly	Lazy generation pattern – seats created on first request via get_or_create_seats, then cached in DB
Gender-based seating constraints	Adjacent seat gender rules required complex pairing logic	Custom apply_gender_rules function processes seats in pairs, marking allowed_gender attribute dynamically
Concurrent seat booking race conditions	Two users selecting the same seat simultaneously could cause double-booking	Database-level is_available flag checked atomically within a single transaction; seat marked unavailable before commit
Cross-origin API communication	React (port 5173) and Flask (port 5000) on different origins	Flask-CORS configured with explicit origin whitelist and credentials support
Secure password transmission	Plain-text passwords over HTTP are vulnerable to interception	Client-side base64 encoding before transmission; server-side Werkzeug PBKDF2 hashing with salt
Payment failure recovery	Network issues during Razorpay checkout could leave bookings in limbo	Payment verification callback confirms booking only after successful payment; modal dismiss and failure handlers redirect gracefully
Email delivery without blocking API	Synchronous email sending would add 2-3s latency to API responses	Asynchronous email dispatch using Python threading; fire-and-forget pattern for non-critical notifications

10. Conclusion

BusBooking is a comprehensive, production-ready bus ticket reservation system that addresses the core pain points of intercity bus travel in India. Through a carefully designed architecture combining React's component model with Flask's lightweight API framework and MySQL's relational data integrity, we have delivered a system that is both performant and maintainable.

The application demonstrates proficiency across the full development stack – from crafting premium glassmorphism UIs with Framer Motion animations to implementing secure JWT authentication, Razorpay payment integration, and asynchronous email notifications. The gender-based seating algorithm and lazy seat generation pattern showcase thoughtful solutions to domain-specific challenges.

Key achievements of this project include:

- End-to-end booking flow covering search, seat selection, payment, and ticket management
- Secure authentication system with login alerts, password reset, and token-based session management
- Production-grade payment integration with proper error handling and recovery flows
- Responsive, animated UI that delivers a premium user experience across all devices
- Clean, modular codebase with separated concerns enabling easy future enhancements

Future enhancements could include real-time seat updates via WebSockets, admin dashboard for operators, multi-language support, and native mobile applications using React Native.